Chapter 6. Composing functions

This chapter covers

- Using functions for program clarity
- The problem with state
- Using functions to create abstractions
- Techniques for combining functions

In contrast to what you learned about objects, you needn't learn any more features to compose programs with functions. Instead, you must learn how to create your own features by putting together functions using the basic function glue that you learned in chapter 3. This principle applies to trivial examples such as defining an `average` function in terms of `sum` and `divide` functions and to nontrivial examples that you'll see in this chapter. To compose programs with functions, you must learn principles and techniques, not features.

Functions have inputs (arguments) and outputs (return values). In that sense, they're like pipes. You need to know not only the different ways of connecting those pipes but also the principles and techniques you need to do so effectively.

In this chapter you'll learn why it's important to break programs into small, clear functions, and how variables and program state can get in the way of doing that effectively. You'll then learn about using functions to create abstractions to make programs smaller and more manageable. Next, you'll learn about some common patterns of abstraction in combinators and, finally, about how function-composition techniques are used to simplify typical CoffeeScript programs that are heavy on callbacks. Before beginning, be aware that this chapter deals with some abstract ideas and techniques that require close attention. With that in mind, first things first—clarity.